VPPaaS Sprint 1 Report

João Russo
António Palma
Bahareh Bodaghi

May 2026

1 Introduction

This report presents the first sprint implementation of the VPPaaS platform. The project follows a microservices architecture using Quarkus, Kafka, Kong, Camunda, AWS infrastructure, and AI-supported forecasting services.

2 Information Flows

In this section, we present the information flows for the main business processes of the VPPaaS platform. The sequence diagrams show how Camunda coordinates the process, how Kong forwards requests to the correct microservices, and where Kafka is used for event-driven communication. In the diagrams, the notes indicating "Process request" represent internal processing inside a microservice, usually involving database access or business logic execution.

2.1 Prosumer Management Business Process

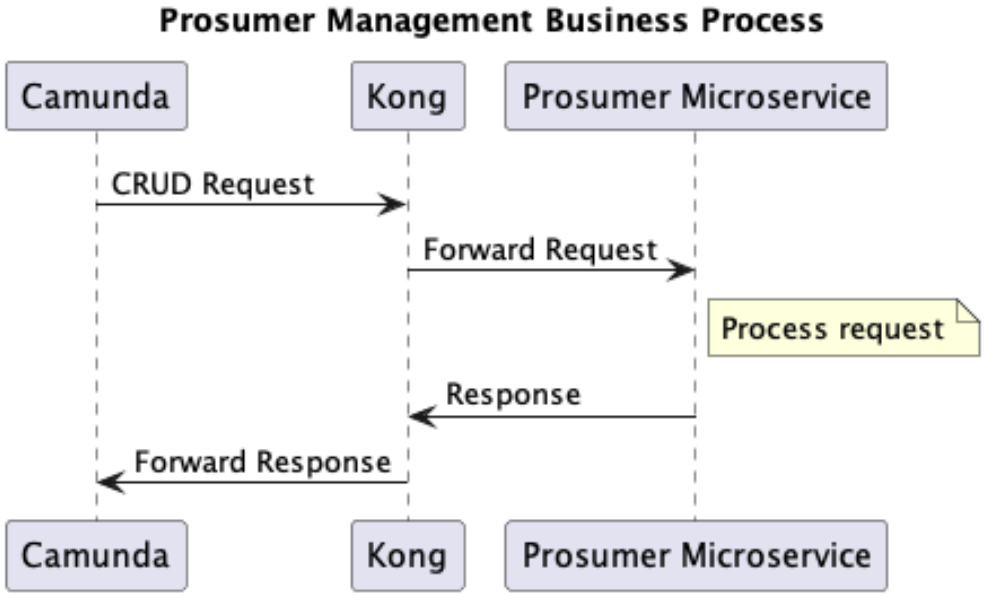

Figure 1: Prosumer Management Business Process

This business process manages CRUD operations related to prosumers. Camunda initiates the request and sends it through Kong, which forwards the request to the Prosumer microservice. The microservice processes the request and returns the response back through Kong to Camunda.

2.2 Utility Operator Management Business Process

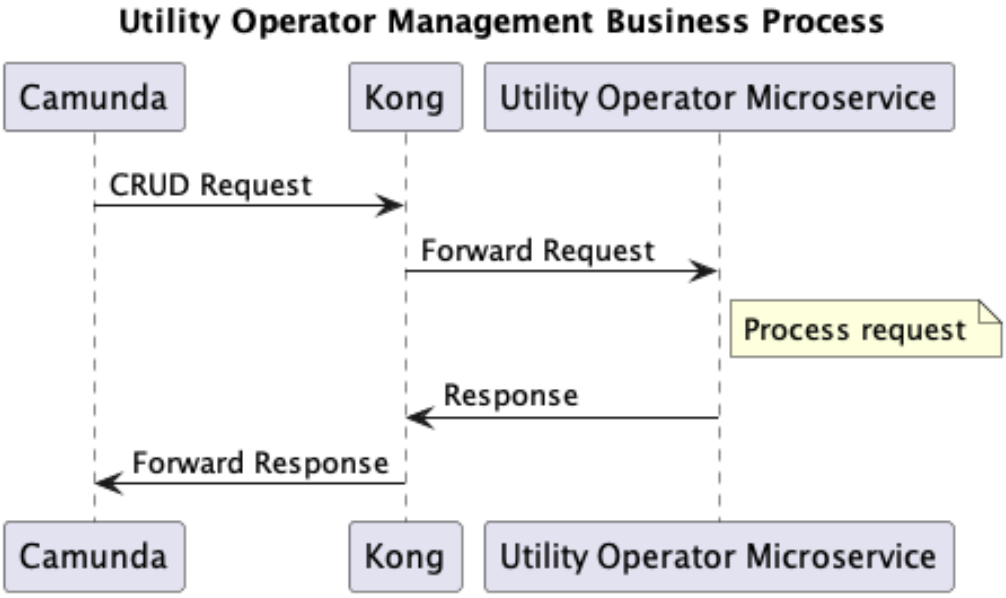

Figure 2: Utility Operator Management Business Process

This business process manages CRUD operations related to utility operators. Camunda sends the request through Kong, which forwards it to the Utility Operator microservice. The microservice processes the request and returns the response through Kong back to Camunda.

2.3 Grid Cell Management Business Process

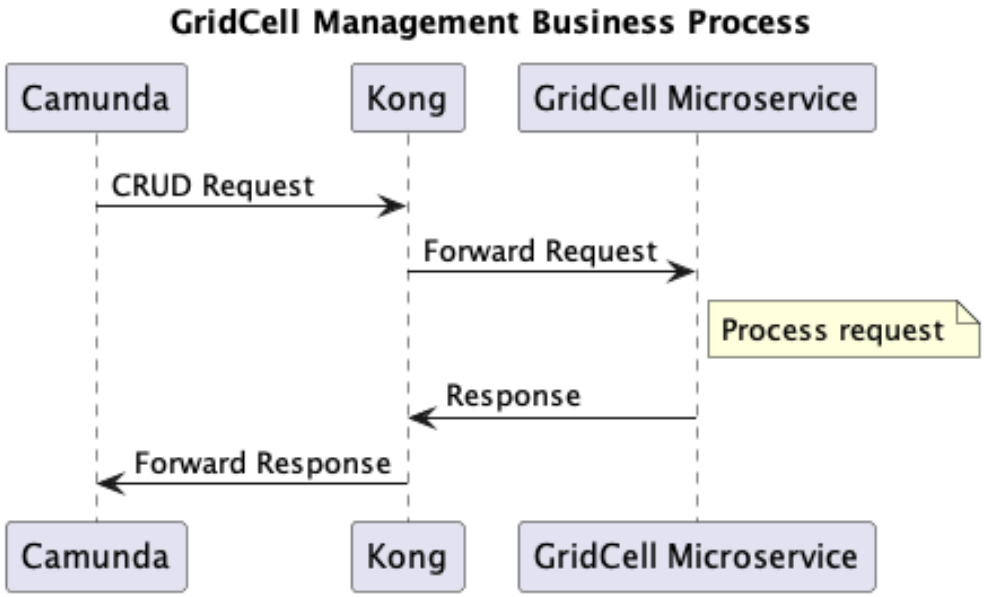

Figure 3: Grid Cell Management Business Process

This business process manages CRUD operations related to grid cells. Camunda sends the request through Kong, which forwards it to the Grid Cell microservice. The microservice processes the request, updates or retrieves the required grid cell information, and returns the response through Kong back to Camunda.

2.4 Asset Link Deletion Business Process

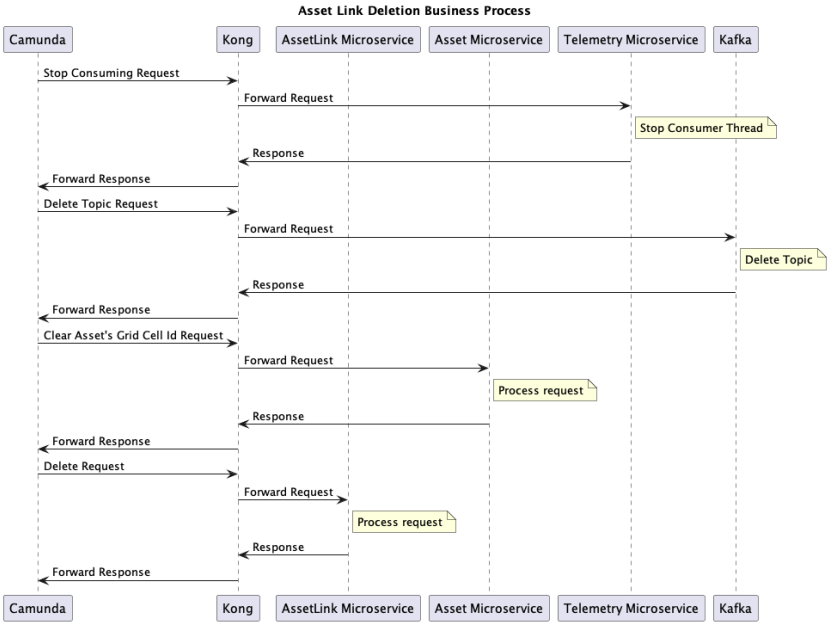

Figure 4: Asset Link Deletion Business Process

This business process manages the deletion of an asset link. When an asset link is removed, the system stops the related telemetry consumption and updates the affected components so that inactive asset links are no longer considered by the platform.

2.5 Asset Link Creation and Reading Business Process

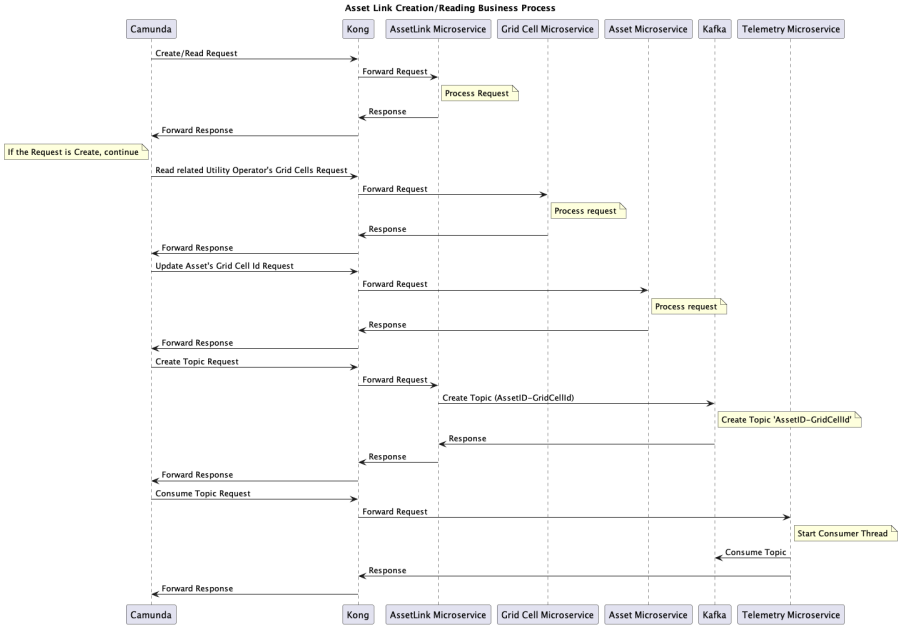

Figure 5: Asset Link Creation and Reading Business Process

This business process manages the creation and reading of asset links. When an asset link is created, the process coordinates the Asset Link, Grid Cell, Asset, Kafka, and Telemetry microservices. The flow creates or reads the asset link, updates the related asset information, creates the required Kafka topic, and asks the Telemetry microservice to consume that topic.

2.6 Asset Management Business Process

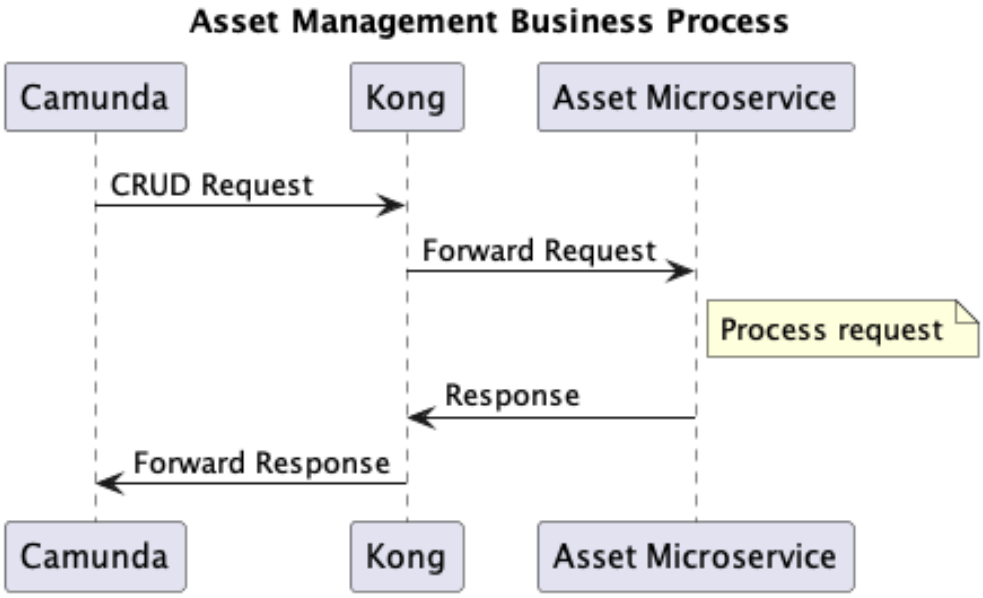

Figure 6: Asset Management Business Process

This business process manages CRUD operations related to assets, such as batteries, solar panels, and EV chargers. Camunda sends the request through Kong, which forwards it to the Asset microservice. The microservice processes the request and returns the response through Kong back to Camunda.

2.7 Telemetry Ingestion Business Process

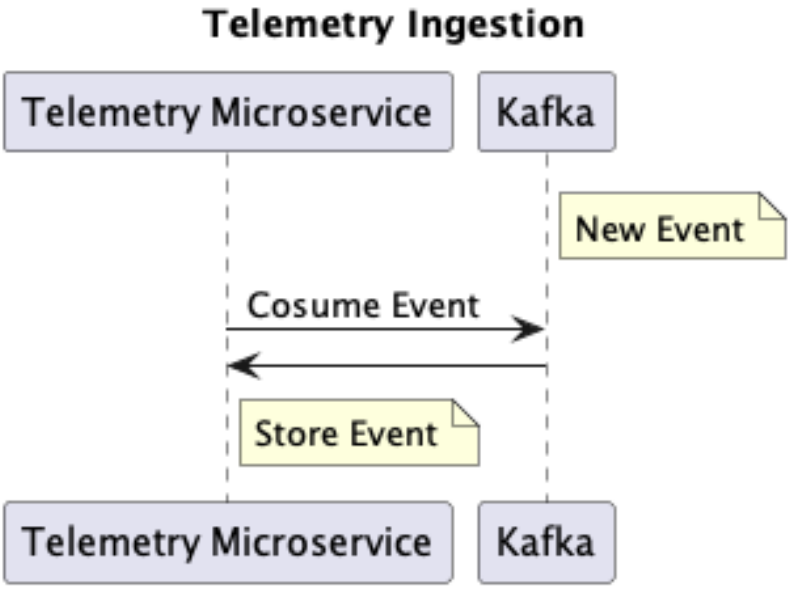

Figure 7: Telemetry Ingestion Business Process

This business process handles telemetry ingestion. Telemetry data is produced to Kafka topics and consumed by the Telemetry microservice. The consumed data represents the real-time state of assets, such as batteries, solar panels, or EV chargers, and can later be used by analytics, flexibility emission, and grid balancing processes.

2.8 Flexibility Emission Business Process

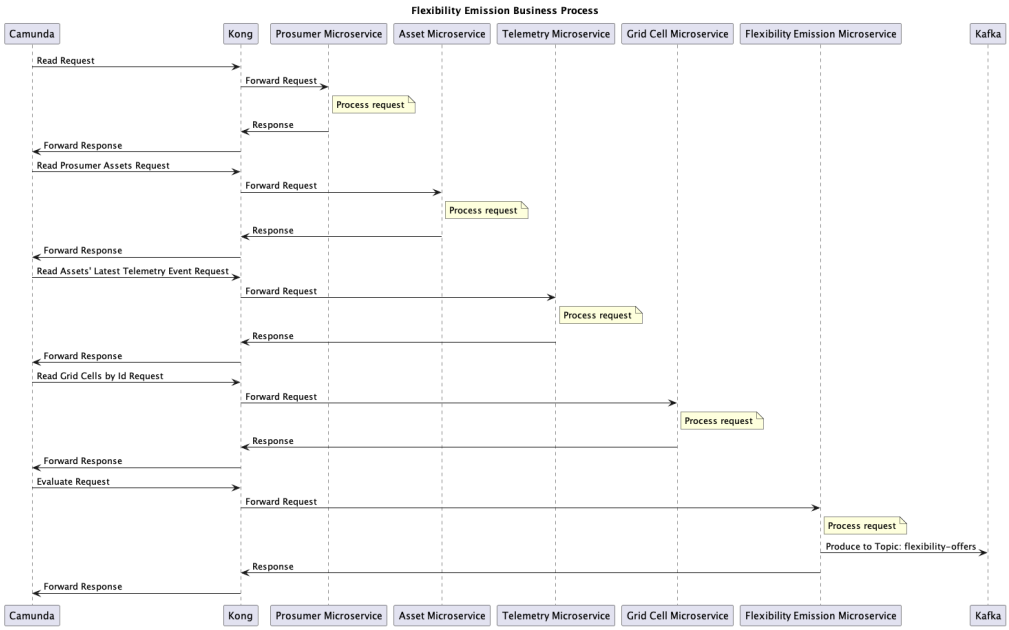

Figure 8: Flexibility Emission Business Process

This business process analyzes telemetry and asset data to decide whether a flexibility event should be emitted. When the required conditions are met, the system creates a flexibility emission event and makes it available to the rest of the platform.

2.9 Grid Balancing Recommendation Business Process

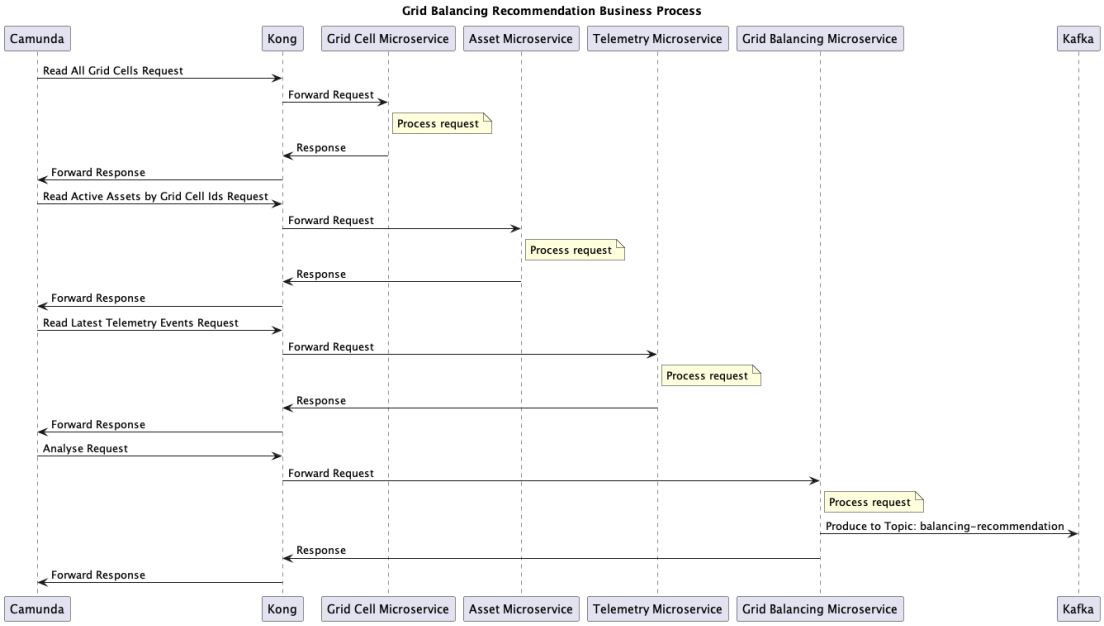

Figure 9: Grid Balancing Recommendation Business Process

This business process analyzes the state of different grid cells and produces recommendations to balance the grid. If one grid cell is under stress and another has available capacity, the system can recommend shifting load between them.

2.10 Energy Analytics Business Process

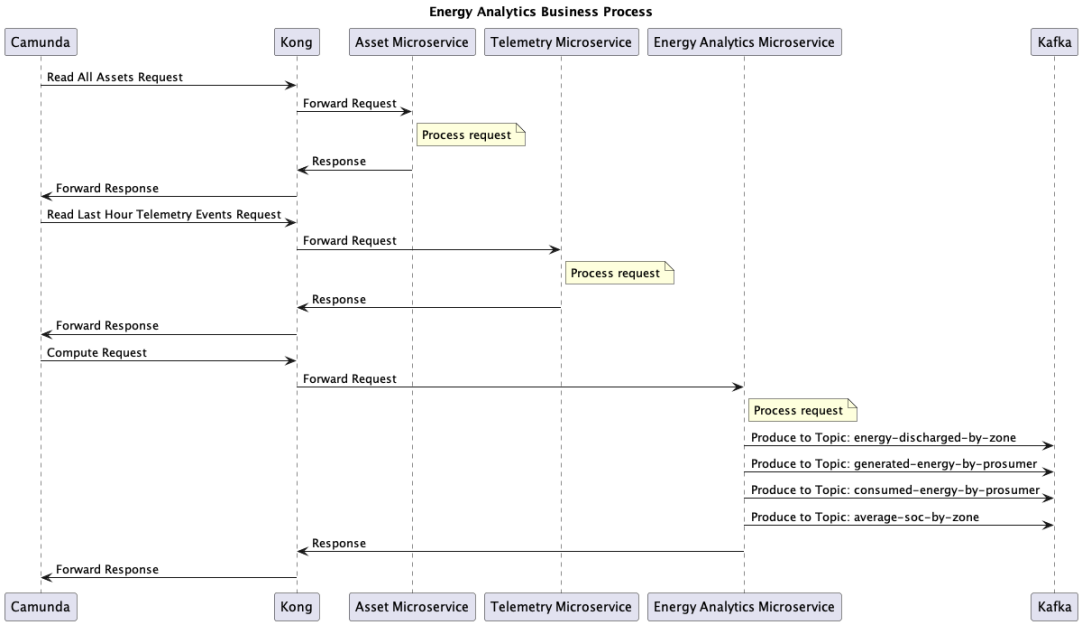

Figure 10: Energy Analytics Business Process

This business process produces energy analytics based on the data collected by the platform. It aggregates information such as energy generated, energy consumed, energy discharged, or average state of charge, supporting later analysis and decision-making in the VPPaaS platform.

3 System Implementation

3.1 Kafka

The Kafka cluster is deployed through Terraform under the `terraform/kafka` directory. The cluster is composed of three Kafka brokers, because the Terraform variable `nBroker` is set to 3. Each broker is deployed as a separate AWS EC2 instance. This improves availability because the failure of one EC2 instance does not necessarily stop the whole Kafka cluster.

The Kafka configuration script sets the default number of partitions to the total number of brokers. Since the project uses three brokers, each topic is created with three partitions by default. The default replication factor is also set to three, meaning that each partition is replicated across the three brokers. In this configuration, each partition has one leader replica and follower replicas on the remaining brokers.

Kafka is mainly used for telemetry and event-based communication. When an asset link is created, the system creates a Kafka topic associated with that asset and grid cell. Telemetry data is then produced to that topic and consumed by the Telemetry microservice.

3.2 RDS

Amazon RDS is used as the managed relational database service. The microservices use MySQL databases hosted on AWS RDS for persistent storage.

Database access is configured through Quarkus datasource properties and, during deployment, the database URL is passed to the containers as an environment variable. This allows the same microservice image to be reused across different environments.

3.3 Ollama

Ollama is used as the AI service integration for flexibility forecasting support. The architecture allows Camunda to send requests directly to the Ollama API instead of using an intermediate Quarkus microservice.

The deploy script exposes Ollama through port `11434`, using the `/api/generate` endpoint. The system sends telemetry and historical operational data to Ollama in order to support flexibility forecasting decisions.

3.4 Microservices

The platform follows a Quarkus-based microservices architecture, where each business domain is isolated into its own independent service to allow scalability and separation of responsibilities. Services are packaged using Maven, containerised as Docker images, pushed to Docker Hub, and deployed on AWS EC2 instances.

3.4.1 Overview

The current microservices are:

- **Asset** — manages physical assets (batteries, solar panels, EV chargers) associated with prosumers and grid cells.
- **AssetLink** — connects assets to grid cells and coordinates Kafka topic creation and telemetry subscriptions.
- **EnergyAnalytics** — aggregates telemetry data to compute energy metrics such as discharge, generation, consumption, and average state of charge.
- **FlexibilityEmission** — evaluates battery telemetry during peak hours and emits flexibility events signalling availability or unavailability.
- **GridBalancingRecommendation** — analyses grid cell conditions and produces energy transfer recommendations between cells.
- **GridCell** — manages electrical grid cell information used to organise assets geographically and operationally.
- **Prosumer** — manages prosumers, representing users capable of both producing and consuming energy.
- **Telemetry** — dynamically consumes telemetry data from Kafka topics and stores operational records for downstream processing.
- **UtilityOperator** — manages utility operator information associated with grid operation activities.

3.4.2 Implementation

Asset The following table summarises the implemented microservice endpoints and their responsibilities.

Microservice	Method	Endpoint	Description
`Asset`	GET	`/Asset`	Get all assets
	GET	`/Asset/{id}`	Get asset by ID
	POST	`/Asset`	Create asset
	POST	`/Asset/{id}`	Update asset
	DELETE	`/Asset/{id}`	Delete asset
	GET	`/Asset/active/by-grid-cell-ids`	Get active assets by grid cell IDs
	GET	`/Asset/active/by-prosumer/{id}`	Get active batteries by prosumer
`AssetLink`	GET	`/AssetLink`	Get all asset links
	GET	`/AssetLink/{id}`	Get asset link by ID
	GET	`/AssetLink/{idProsumer}/{idUtilityOperator}`	Get asset link by prosumer and operator
	POST	`/AssetLink`	Create asset link
	DELETE	`/AssetLink/{id}`	Delete asset link
	POST	`/AssetLink/topic/{assetId}/{gridCellId}`	Create Kafka topic for asset link
	DELETE	`/AssetLink/topic/{assetId}/{gridCellId}`	Delete Kafka topic for asset link
`Energy Analytics`	GET	`/EnergyAnalytics`	Get all analytics records
	POST	`/EnergyAnalytics/analyse`	Run energy analysis and store results
`Flexibility Emission`	GET	`/FlexibilityEmission`	Get all flexibility events
	POST	`/FlexibilityEmission/evaluate`	Evaluate and emit flexibility events
`GridBalancing Recommendation`	GET	`/GridBalancingRecommendation`	Get all balancing recommendations
	POST	`/GridBalancingRecommendation/balance`	Calculate and emit balancing recommendations
`GridCell`	GET	`/GridCell`	Get all grid cells

Continued on next page

Continued from previous page

Microservice	Method	Endpoint	Description
	GET	`/GridCell/{id}`	Get grid cell by ID
	POST	`/GridCell`	Create grid cell
	PUT	`/GridCell/{id}`	Update grid cell
	DELETE	`/GridCell/{id}`	Delete grid cell
	GET	`/GridCell/by-ids`	Get grid cells by IDs
	GET	`/GridCell/by-operator-ids`	Get grid cells by operator IDs
`Prosumer`	GET	`/Prosumer`	Get all prosumers
	GET	`/Prosumer/{id}`	Get prosumer by ID
	POST	`/Prosumer`	Create prosumer
	PUT	`/Prosumer/{id}/{name}/{FiscalNumber}/{location}`	Update prosumer
	DELETE	`/Prosumer/{id}`	Delete prosumer
`Telemetry`	GET	`/Telemetry`	Get all telemetry events
	GET	`/Telemetry/last-hour`	Get telemetry from the last hour
	GET	`/Telemetry/asset/{assetId}/around/{timestamp}`	Get telemetry around a timestamp
	POST	`/Telemetry/latest`	Get latest telemetry for asset/grid cell pairs
	POST	`/Telemetry/consume`	Start Kafka consumer for a topic
	POST	`/Telemetry/stop`	Stop Kafka consumer for a topic
`Utility Operator`	GET	`/UtilityOperator`	Get all utility operators
	GET	`/UtilityOperator/{id}`	Get utility operator by ID
	POST	`/UtilityOperator`	Create utility operator
	PUT	`/UtilityOperator/{id}`	Update utility operator
	DELETE	`/UtilityOperator/{id}`	Delete utility operator

3.4.3 Design Decisions

Kafka Topic Auto-Creation Automatic Kafka topic creation is disabled in production to prevent silent misconfigurations, enforce correct partitioning and replication settings, and maintain naming governance. Topics are instead provisioned explicitly via a dedicated endpoint.

No Update Operation on AssetLink An update operation for `AssetLink` was intentionally omitted. Since updating a link would require deleting and recreating the underlying Kafka topic, the API exposes this honestly through separate delete and create endpoints rather than presenting a misleading update abstraction.

Foreign Key Constraints Not Enforced Each microservice owns its own database, making database-level referential integrity across services architecturally impossible. Consistency is instead maintained at the application layer through inter-service API calls.

Dynamic Consumer Startup and Offset Behaviour The Telemetry service dynamically starts and stops Kafka consumers as `AssetLink`s are created and deleted, with automatic recovery on restart. Kafka's committed offset tracking ensures no events are lost during restarts, though a small number may be re-processed if they were consumed but not yet committed before a failure.

Asset Type Filtering in Flexibility Emission Only assets of type `BATTERY` are included in flexibility calculations, as they are the only fully dispatchable resource in a VPP context. Asset types such as `SOLAR` and `EV_CHARGER` are excluded due to non-dispatchability or being outside the current implementation scope.

Flexibility Forecasting and AI Integration The Flexibility Forecasting step is handled entirely by Camunda, which calls `Llama` directly. No changes were made on our end regarding AI prompting, as this interaction is fully orchestrated by Camunda.

4 Tests

The project includes automated tests implemented inside the Quarkus microservices using JUnit, `@QuarkusTest`, and RestAssured. The tests are located under each microservice's `src/test/java` directory, mostly in `org/acm`, with Telemetry tests in `org/acme`.

Current test suites include:

- `AssetResourceTest.java`
- `AssetLinkResourceTest.java`
- `KafkaTopicServiceTest.java`
- `EnergyAnalyticsResourceTest.java`
- `FlexibilityEmissionResourceTest.java`
- `GridBalancingRecommendationResourceTest.java`
- `GridCellResourceTest.java`
- `ProsumerResourceTest.java`
- `TelemetryResourceTest.java`
- `DynamicTopicConsumerTest.java`
- `KafkaConsumerServiceTest.java`
- `TopicSubscriptionRecoveryServiceTest.java`
- `UtilityOperatorResourceTest.java`

The tests validate REST endpoints, database interactions, and Kafka-related operations. The Asset microservice tests illustrate the general testing approach used throughout the platform.

Before each Asset test, the database table is cleaned and populated with sample records. The tests then validate endpoints such as:

- `GET /Asset`
- `GET /Asset/{id}`
- `POST /Asset`
- `POST /Asset/{id}`
- `DELETE /Asset/{id}`

The tests verify both successful and error scenarios, including validation of HTTP status codes and response contents.

For database testing, the project uses Quarkus Dev Services with MySQL 8 test containers. This allows the tests to run in an isolated environment without depending on the production AWS RDS instance.

Kafka-related tests validate dynamic topic creation, Kafka consumer behaviour, telemetry subscriptions, and topic recovery mechanisms. Some Kafka behaviour is tested using mocks or local configuration rather than a real broker. These tests are especially important because Kafka is responsible for the event-driven communication layer of the platform.

To run the tests for a microservice, the developer can enter the microservice directory and execute:

```
./mvnw test
```

5 Project Structure and Deployment

The project is organised into four main areas: business-process diagrams, Quarkus microservices, Terraform infrastructure code, and deployment support scripts. This separation makes the repository easier to navigate and helps distinguish application logic from infrastructure and operational tooling.

5.1 Top-Level Structure

```
project/
|-- diagrams/                       # PlantUML sequence diagrams
|-- microservices/                  # Quarkus microservices
|   |-- Asset/
|   |-- AssetLink/
|   |-- EnergyAnalytics/
|   |-- FlexibilityEmission/
|   |-- GridBalancingRecommendation/
|   |-- GridCell/
```

```
|   |-- Prosumer/
|   |-- Telemetry/
|   `-- UtilityOperator/
|-- terraform/
|   |-- kafka/                      # Kafka cluster deployment
|   |-- rds/                        # RDS database deployment
|   `-- microservices/              # EC2 deployment per microservice
|-- logs/                           # Terraform deployment logs
|-- deploy.sh                       # Full deployment script
|-- undeploy.sh                     # Full teardown script
|-- access.sh                       # Docker credentials setup
`-- addresses.sh                    # Retrieve deployed service addresses
```

The `diagrams/` directory contains the PlantUML sources used to document the platform's information flows. The `microservices/` directory groups the Quarkus services by business domain. The `terraform/` directory contains the infrastructure-as-code modules used to provision Kafka, RDS, and the EC2 instances that host the services. Finally, the shell scripts at the project root support deployment, teardown, authentication, and discovery of deployed service addresses.

5.2 Microservice Internal Structure

Each microservice follows a shared internal layout based on Quarkus and Maven conventions, which keeps the services consistent and easier to maintain.

```
<ServiceName>/
|-- pom.xml                         # Maven build configuration
|-- src/
|   |-- main/
|   |   |-- java/org/acm(e)/        # REST resources and business logic
|   |   `-- resources/
|   |       `-- application.properties
|   `-- test/
|       `-- java/org/acm(e)/        # JUnit tests
`-- README.md
```

The `pom.xml` file defines dependencies and build configuration. The `src/main/java` directory contains the REST resources, service classes, and domain logic, while `src/main/resources/application.properties` stores runtime configuration. The `src/test/java` directory contains the automated test suites, and the `README.md` file documents service-specific usage and implementation details.

Deployment and configuration are managed through Terraform modules and shell scripts. Infrastructure parameters such as AWS resources, Kafka broker count, database connection settings, and service endpoints are configured through Terraform variables and Quarkus `application.properties` files.

5.3 Deployment Procedure

The platform deployment process is performed through shell scripts and Terraform modules.

1. Configure AWS credentials in:

   ```
   terraform/.aws/.credentials
   ```

2. Create `access.sh` from `access.sh.example` and define the Docker Hub credentials:

   ```
   export DockerUsername=
   export DockerPassword=
   ```

3. Execute the authentication script:

   ```
   ./access.sh
   ```

4. Deploy the infrastructure and microservices:

   ```
   ./deploy.sh
   ```

5. Retrieve the deployed service addresses:

   ```
   ./addresses.sh
   ```

6. Remove the deployed infrastructure:

   ```
   ./undeploy.sh
   ```